

# Real-Time Scheduling for Medical Monitoring

FIFO vs. EDF on Embedded Linux

**Author:** Oumaima Yaakoubi

**Program:** Industrial Maintenance and Instrumentation (IMI), INSAT

**Date:** Mai, 2026

## Abstract

This report presents a real-time task scheduler for medical sensor monitoring, implemented in C on Ubuntu Linux using POSIX threads. Two scheduling algorithms, FIFO and EDF, are compared under two load scenarios: a normal four-sensor configuration and an extreme fifteen-sensor configuration where CPU demand reaches twice the available capacity. Under normal load, both algorithms succeed completely. Under overload, EDF reduces missed deadlines by 27% and protects vital sensors such as ECG at the expense of less critical ones.

## 1 Introduction: Why Real-Time Scheduling Matters

A real-time system is not a fast system. It is a system that guarantees a response before a deadline. The classic example is an airbag: it must fire within milliseconds, not as fast as possible but before a fixed moment. Speed is secondary. Predictability is everything.

Medical monitoring is a similar case. In an intensive care unit, several sensors track the patient at the same time: heart rate, oxygen saturation, blood pressure, temperature. Each reading has a deadline by which it must be processed. If the system cannot keep up, the readings that matter most may arrive too late.

This project asks a precise question: when the CPU is overloaded and cannot process every reading on time, which scheduling algorithm protects the patient best?

### Key Insight

Under normal load, any reasonable scheduler works. The difference only appears when the system is overloaded and forced to choose between tasks.

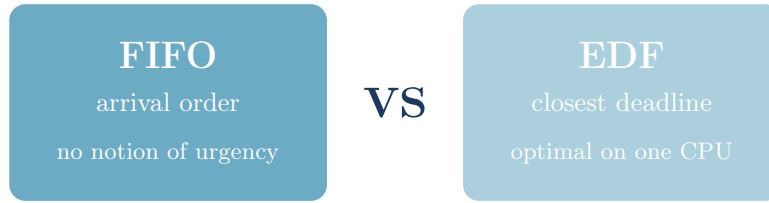
## 2 The Two Algorithms

### 2.1 FIFO (First In First Out)

FIFO processes readings in the order they arrive, like a supermarket queue. It is simple and predictable, but it has no notion of urgency. An ECG reading due in 20 ms can wait behind a temperature reading due in 200 ms.

### 2.2 EDF (Earliest Deadline First)

EDF always processes the reading whose deadline is closest. It behaves like hospital triage: the most urgent case goes first. Liu and Layland proved in 1973 that EDF is optimal on a single processor, meaning that if any algorithm can meet all deadlines, EDF can.



**Figure 1:** The two scheduling policies compared in this project.

### 3 System Architecture

The system is a single C program that simulates a real-time operating system on top of Linux. It has three layers.

#### 3.1 The Sensors

Each sensor is modelled as a structure containing its name, period, deadline, execution time and priority. Two configurations are defined:

- Phase 1: four sensors (ECG, SpO2, blood pressure, temperature), total CPU load around 75%.
- Phase 2: fifteen sensors, total CPU load around 200%, deliberately overloading the CPU.

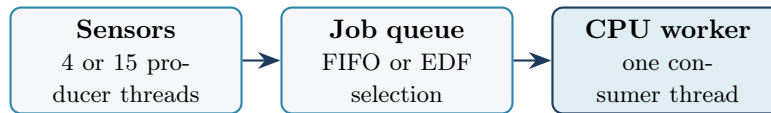
Each sensor has its own producer thread that generates a new reading every period.

#### 3.2 The Scheduler

A shared job queue holds all pending readings. Each reading carries a release time, an absolute deadline and a sequence number. The scheduler selects the next job to run using either FIFO or EDF, depending on the command-line argument. The selection logic lives in two small files, one per algorithm.

#### 3.3 The CPU Worker

A single consumer thread represents the CPU. It pulls one job at a time from the queue, simulates the processing time with a short sleep, and records the result.



**Figure 2:** The producer-queue-consumer architecture of the scheduler.

## 4 Results

#### 4.1 Phase 1: Normal Load

Under 75% CPU load, both algorithms succeed completely. Every reading is processed before its deadline.

**Table 1:** Phase 1 results. No deadline is missed by either algorithm.

| Algorithm | Total executions | Deadline misses |
|-----------|------------------|-----------------|
| FIFO      | 4756             | 0               |
| EDF       | 4768             | 0               |

**Interpretation**

When the CPU has spare capacity, the choice of algorithm is irrelevant. Both FIFO and EDF meet every deadline. The stress test is needed to reveal the real difference.

**4.2 Phase 2: Extreme Overload**

Under 200% CPU load, the system cannot process everything. The scheduler must decide which readings to run and which to delay. This is where the two algorithms diverge.

**Table 2:** Phase 2 results. EDF reduces missed deadlines by 27%.

| Metric              | FIFO    | EDF     | Gain      |
|---------------------|---------|---------|-----------|
| Total executions    | 3399    | 3432    | +33       |
| Total misses        | 447     | 325     | -122      |
| Success rate        | 86.85%  | 90.53%  | +3.68 pts |
| ECG misses          | 120     | 98      | -22       |
| Average ECG latency | 5.42 ms | 4.70 ms | -0.72 ms  |

**Interpretation**

EDF sacrifices low-priority sensors (temperature, glucose) to protect vital ones. Under EDF, temperature and glucose show 0% misses, not because they are prioritised but because they are systematically pushed to the back of the queue, where they wait until the CPU has free time. The important result is that EDF saves 22 ECG readings that FIFO would have dropped. In a real ICU, those 22 readings could be the moment an arrhythmia appears.

**5 Discussion****5.1 What Worked**

- Both algorithms run correctly, with results recorded per sensor and written to CSV.
- The producer-consumer architecture correctly models a shared CPU under contention.
- Under overload, EDF measurably protects the most urgent sensors.

**5.2 Limitations****Important Limitations**

- Simulation only. Sensor readings are generated values, not real physiological signals.
- The CPU processing time is simulated with a sleep, not real computation.
- The system was tested on a virtual machine, so timing is looser than on real embedded hardware.

### 5.3 Lessons Learned

#### Why Extreme Testing Matters

In normal conditions, every algorithm works. It is only under deliberate overload that the weaknesses become visible. This is true in real-time systems, in machine learning and in reliability engineering.

#### Why EDF Is Optimal in Theory and in Practice

Liu and Layland's 1973 proof says that EDF meets every deadline whenever any algorithm can. Our experiments confirm this. The theoretical property translates directly into saved ECG readings under overload.

## 6 Conclusion

This project demonstrates that scheduling policy has a measurable effect on the reliability of a real-time medical monitoring system. Under normal load, FIFO and EDF behave identically. Under overload, EDF reduces missed deadlines by 27% and protects the most critical sensors. For a critical-care application, this difference matters. The project also shows that Linux, with POSIX threads and real-time priorities, is capable of hosting such a system without needing a dedicated real-time operating system.

## 7 Tools and Environment

**Table 3:** Tools and technologies used in this project.

| Category         | Tool                       | Role in the project                   |
|------------------|----------------------------|---------------------------------------|
| Operating system | Ubuntu Linux (Virtual-Box) | Execution environment                 |
| Language         | C                          | All source code                       |
| Compiler         | GCC                        | Compiles the project                  |
| Build system     | Make                       | Automates compilation via Makefile    |
| Concurrency      | POSIX threads              | One thread per sensor, one CPU worker |
| Scheduling       | SCHED_FIFO (Linux)         | Real-time priority for the CPU worker |
| Storage          | CSV files                  | Per-sensor metrics for analysis       |
| Analysis         | LibreOffice Calc           | Visualising the CSV results           |

**Oumaima Yaakoubi**

Instrumentation and Intelligent Systems, INSAT, Tunisia

[linkedin.com/in/oumaima-yaakoubi](https://www.linkedin.com/in/oumaima-yaakoubi) • [github.com/o-Yaakoubi](https://github.com/o-Yaakoubi)